



Sviluppo di una Pipeline di Telemetria per Autobus

Dario Macchi

Indice

Introduzione.....	3
Capitolo 1. Scelte progettuali.....	4
1.1. Sensoristica e metriche.....	4
1.1.1. Metriche.....	4
1.1.2. Formato invio metriche.....	7
1.1.3. Frequenza raccolta metriche ed eventi.....	8
1.2. Sistema centralizzato.....	9
1.2.1. Posizionamento.....	9
1.2.2. Processing.....	12
Capitolo 2. Strumenti e tecnologie utilizzate.....	13
2.1. Protocollo di comunicazione - MQTT.....	13
2.2. Ingestion - Apache Kafka.....	16
2.3. Processing - Apache Flink.....	19
2.4 Storage e Visualizzazione - MongoDB e Grafana.....	22
Capitolo 3. Analisi del funzionamento.....	24
3.1. Produzione dati.....	25
3.2. Integration layer.....	28
3.3. Ingestion & Storage layers.....	30
3.4. Processing layer.....	32
3.5. Visualization layer.....	36
Conclusioni e sviluppi futuri.....	40
Bibliografia.....	41

Introduzione

La telemetria è la raccolta e trasmissione automatizzata di dati e misurazioni da fonti distribuite verso un sistema centrale per il monitoraggio, l'analisi e l'ottimizzazione delle risorse. La telemetria svolge un ruolo fondamentale in vari settori, tra cui spicca l'automotive, offrendo alle organizzazioni preziosi report sulle prestazioni del sistema, sull'efficienza operativa e altro ancora. Nei settori che si basano su asset fisici, come quello dei trasporti, le compagnie utilizzano la telemetria per acquisire misurazioni come temperatura e pressione atmosferica, sfruttando le capacità offerte da strumenti e sensori in grado di raccogliere dati del mondo reale e inviarli a un repository centrale [IBM25].

L'attività svolta ha come obiettivo la creazione, per conto di una società di trasporto pubblico, di un sistema di monitoraggio in tempo reale di autobus, in grado di utilizzare i dati di telemetria per migliorare l'efficienza operativa. Inoltre le metriche raccolte dovranno essere trasmesse dai veicoli a una piattaforma centralizzata tramite un protocollo di comunicazione, piattaforma centralizzata che si occuperà di Ingestion, Processing, Storage e Visualizzazione dei dati. Pertanto si rende necessaria la progettazione dell'architettura generale, comprendendo ragionamenti sui collegamenti tra gli autobus e la soluzione centralizzata, e sul posizionamento della stessa soluzione centralizzata; la selezione delle tecnologie più adatte allo scenario; l'implementazione di un prototipo, ossia una Proof of Concept, in grado di dimostrare quanto necessario alla società di trasporto pubblico.

Nel Capitolo 1 verranno introdotte le scelte progettuali, a partire dalla sensoristica fino a toccare il sistema centralizzato. Nel Capitolo 2 si introdurranno tutte le tecnologie utilizzate nel processo di sviluppo e le relative scelte, a partire dal protocollo di comunicazione fino ad arrivare al servizio di visualizzazione. Nel Capitolo 3 si tratterà l'operato vero e proprio del sistema sviluppato, affrontando ogni strato del sistema realizzato. Al termine dell'elaborato si trarranno le conclusioni e si presenteranno alcuni possibili sviluppi futuri.

Capitolo 1. Scelte progettuali

Buona parte dello sforzo per la realizzazione della soluzione di telemetria, è stata spesa nello studio della realtà in cui si cala un'architettura di questo genere, per avere la sicurezza che la pipeline messa in piedi possa avere un'utilità effettiva in uno scenario reale.

1.1. Sensoristica e metriche

Si presentano le scelte progettuali riguardanti la sensoristica e le metriche della soluzione di pipeline di telemetria realizzata.

1.1.1. Metriche

I sistemi Automatic Vehicle Monitoring (AVM) prevedono diverse metriche con cui è interessante interagire e che necessitano la raccolta [WIK01]:

- **Global Positioning System (GPS).** La raccolta della posizione in tempo reale, da comunicare al sistema centralizzato di raccoglimento di tutti i dati provenienti dalla flotta di autobus monitorati, è determinante nel momento in cui sia necessario l'invio di unità di soccorso verso l'autobus, a seguito ad esempio della ricezione dell'allarme associato al panic button. La posizione è stata intesa come insieme di coordinate, in particolare latitudine e longitudine, a copertura di un unico percorso, calcolato come terreno di prova per la soluzione realizzata.
- **Velocità.** Risulta utile da mantenere sotto controllo nell'eventualità di condizioni problematiche, associate all'autobus stesso oppure all'ambiente circostante.
- **Pressione degli pneumatici.** Importante monitorare questa metrica per evitare eventuali problemi alla flotta di autobus, dovuti a una pressione degli pneumatici inappropriata alla marcia di veicoli dotati di peso ingente.
- **Usura dei freni.** L'impianto frenante è uno degli apparati fondamentali nel funzionamento meccanico e idraulico di tutti i veicoli, di conseguenza è necessario mantenere sotto controllo l'usura di questo impianto, per evitare

che condizioni critiche che lo riguardano possano condurre a problematiche gravi, gravità che viene aumentata ancora di più dall'utilizzo di questi veicoli come servizio pubblico per il trasporto di persone.

- **Stato del motore.** Lo stato del motore deve essere, chiaramente, tenuto sotto controllo per ragioni del tutto simili al monitoraggio dell'usura dei freni. Tra le motivazioni che portano all'usura del motore è sicuramente presente il surriscaldamento dello stesso, tant'è che la pipeline di telemetria sviluppata osserva la condizione del motore per ogni autobus, e la valutazione associata dipende proprio dall'eventuale surriscaldamento.
- **Livello di carica delle batterie / livello di carburante.** La soluzione architettata prevede l'osservazione del livello di carburante o del livello di carica della batteria, in base alla tipologia di motorizzazione del mezzo monitorato, per evitare problemi di consumi eccessivamente rapidi.
- **Temperatura batterie.** Risulta importante per i veicoli della tipologia consona, ossia autobus con motorizzazione ibrida o elettrica, controllare la temperatura della batteria, per evitare guasti annessi a temperature della stessa eccessivamente elevate.
- **Consumo di carburante.** La tale metrica, associata alla precedente di livello del carburante, permette un controllo incrociato sui consumi dei mezzi in marcia, per un monitoraggio dotato di un maggiore grado di certezza.
- **Numero di passeggeri a bordo.** Presente in commercio un modulo conta passeggeri che permette di raccogliere la taluna metrica. Il suo utilizzo permette alla centrale operativa di prendere decisioni su orari e frequenze, e rendere il servizio offerto maggiormente aderente alla domanda dei fruitori del trasporto pubblico [PLU01].
- **Dati ambientali.** Le condizioni atmosferiche, come temperatura dell'aria e umidità circostante, sono in grado di determinare condizioni del manto stradale che possono rendere difficoltosa la marcia. Il monitoraggio di queste ultime può risultare determinante, dato che informando la centrale di controllo delle attuali condizioni, possono essere apportate modifiche ai percorsi dei bus ove necessario.

- **Spia del motore.** L'osservazione dello stato dell'allarme motore, può comandare una scelta tempestiva da parte della centrale di controllo di fermare l'autobus oppure verificare l'evoluzione della situazione, sulla base dello storico delle problematiche di un determinato veicolo.
- **Panic button.** Un segnale di emergenza immediato inviato alla centrale operativa, in seguito alla pressione dell'apposita strumentazione di bordo. L'allarme in questione viene associato alla posizione del mezzo e al numero di passeggeri a bordo, informazioni determinanti per le unità di soccorso tempestivamente allertate da parte del centro di controllo, in seguito alla ricezione del suddetto allarme.

La decisione della raccolta delle metriche sopra elencate è comandata da uno scopo principe dei sistemi AVM, ossia la prevenzione di guasti che possano comportare la fermata dei bus appartenenti alla flotta. Tant'è che, la combinazione di sistemi di questo tipo a manutenzione preventiva, può garantire un tempo di utilizzo dei veicoli molto alto, e contemporaneamente tempistiche di fermata dei mezzi molto basse e, aspetto ancora più importante, previste.

Si noti inoltre che le metriche di spia del motore e di panic button sono allarmi, in quanto tali all'interno della pipeline si interpretano come eventi, di conseguenza possiedono un'accezione differente nel sistema, questo porta risvolti, analizzati in seguito, in alcuni passaggi dell'architettura.

Oltre alle metriche monitorate, all'autobus smart si associano un identificativo e il timestamp, per mantenere i dati discriminati tra loro e ordinati temporalmente. La scelta per l'identificativo è ricaduta sulla targa. Ai fini di mantenere un autobus identificato da una targa sempre della stessa motorizzazione, la scelta progettuale condotta è la disposizione di un pool statico di targhe per ogni tipologia di autobus smart. Quest'ultima conduce a un trade-off, da una parte il vantaggio di mantenere correttamente separati gli autobus tra di loro, dall'altra lo svantaggio di ottenere un numero limitato di autobus per motorizzazione da poter osservare. Il taluno trade-off è associato in maniera evidente alla obbligata simulazione di un sistema di telemetria, in uno scenario reale il problema si potrebbe risolvere grazie alla disposizione di un database, il quale sarebbe in grado di memorizzare tutte le targhe dei veicoli gestiti dall'azienda di trasporto pubblico.

Infine, la discriminazione del tipo di motorizzazione degli autobus smart monitorati implica una evidente predisposizione della soluzione al paradigma Object Oriented Programming (OOP), in particolare alla caratteristica dell'ereditarietà. Di seguito le diverse tipologie di mezzo di trasporto incluse nella pipeline:

- termico;
- ibrido;
- elettrico.

Questa suddivisione permette la raccolta di dati che risultano interessanti solamente nel contesto dell'autobus di una determinata motorizzazione. Quest'ultima scelta progettuale determina risvolti nelle tecnologie dedite alla comunicazione utilizzate e successivamente introdotte, MQTT e Apache Kafka. In particolare consente una suddivisione altamente funzionale della gerarchia dei topic utilizzati.

1.1.2. Formato invio metriche

Il formato di invio delle metriche determina svariati aspetti nella comunicazione, per questa motivazione si sono valutate alcune alternative, tutte fortemente associate a una naturale soluzione per il protocollo di comunicazione. Una prima alternativa è rappresentata dalla scelta del protocollo di comunicazione Constrained Application Protocol (CoAP) con formato di invio metriche binario, che porta a un trade-off, dalla parte vantaggiosa una bassa occupazione di banda e di conseguenza una comunicazione maggiormente lightweight, dalla parte meno vantaggiosa la necessità di convertire il messaggio in un formato differente, dato che il formato binario non costituisce una soluzione human-readable, quindi si renderebbe necessaria una elaborazione sul messaggio a posteriori della comunicazione. La seconda alternativa vagliata è rappresentata dal formato di invio metriche JavaScript Object Notation (JSON), possibilità maggiormente human-readable, abbinato al protocollo di comunicazione Message Queuing Telemetry Transport (MQTT), quest'ultimo possiede la stessa caratteristica lightweight di CoAP, che comporta un basso utilizzo di banda per il trasporto del messaggio, proprio per questo l'apporto dell'header del protocollo può essere considerato trascurabile in dimensione, dimensione che di conseguenza è determinata quasi unicamente dal payload.

A fronte di una maggiore dimensione occupata dal formato di invio metriche JSON, la scelta è ugualmente ricaduta su quest'ultimo, per la leggibilità e la semplicità del formato, che è stata valutata troppo importante. Si noti inoltre che la scelta effettuata tiene in considerazione la rete su cui transita il traffico, dato che non è presente nessuna forma di comunicazione intermedia dei device, che in questo caso sono rappresentati dai moduli degli autobus smart, verso dispositivi gateway, la rete sfruttata potrebbe essere rappresentata in maniera calzante dall'infrastruttura di telecomunicazione cellulare, affatto limitata in termini di throughput, permettendo l'utilizzo di un formato di invio metriche altresì meno leggero rispetto al binario, quale è il JSON.

1.1.3. Frequenza raccolta metriche ed eventi

Si è scelto di dimensionare la frequenza di raccolta delle metriche prodotte dal sistema di telemetria pari a una volta ogni cinque secondi, per ottenere un bilanciamento, e non avere una frequenza troppo bassa o troppo alta di invio metriche. Una frequenza troppo alta, ossia un high rate al secondo, potrebbe comportare la raccolta di troppi dati ogni secondo, il che condurrebbe a considerazioni fuorvianti nelle analisi dei dati. Una frequenza troppo bassa, ossia un low rate al secondo, potrebbe al contrario comportare una scarsa raccolta di dati, il che potrebbe condurre a interventi tardivi nel caso di problematiche ai mezzi in circolazione. Alla luce di ciò il giusto bilanciamento, dopo diverse prove sperimentali, è stato raggiunto alla frequenza di una volta ogni cinque secondi, parametro che in ogni caso risulta facilmente "tunable".

Per quanto concerne la frequenza di raccolta eventi quest'ultima non è stata definita, dato che la loro generazione è casuale e dipendente da alcune distribuzioni di probabilità. In particolare si è scelto di produrre gli eventi di spia del motore e di panic button a partire dalla distribuzione di probabilità Gaussiana, con parametri μ e σ , rispettivamente media e deviazione standard della distribuzione, differenti, nello specifico la distribuzione associata all'evento di spia del motore fa in modo che il suddetto evento abbia maggiore probabilità di accadere rispetto all'evento panic button.

- **Gaussiana per spia del motore.** La scelta per i parametri μ e σ della taluna distribuzione di probabilità è ricaduta su $\mu = 0$ e $\sigma = \frac{1}{2}$, questo consente di ottenere l'evento con una bassa probabilità in un numero alto di esecuzioni, tant'è che la probabilità di estrarre l'evento è di circa 2.28%.
- **Gaussiana per panic button.** La scelta per i parametri μ e σ per questa distribuzione di probabilità è ricaduta su $\mu = 0$ e $\sigma = 0.43$, questo consente di ottenere l'evento con una probabilità ancora inferiore nello stesso numero di esecuzioni, tant'è che la probabilità di estrarre l'evento è mediamente 1.00%.

1.2. Sistema centralizzato

Si presentano le scelte progettuali riguardanti il posizionamento del sistema centralizzato e il processamento effettuato per la soluzione di pipeline di telemetria realizzata.

1.2.1. Posizionamento

Lo studio del posizionamento della soluzione centralizzata ha coinvolto l'analisi di alcune casistiche calate nell'ambito del settore dei trasporti, tra queste il sistema AVM sviluppato da Roma TPL [SMA01], da Leonardo [LEO21], e una simile soluzione realizzata in scala minore da parte del Comune di Ferrara [CRO01]. Il caso di studio riportato riguarda Roma TPL, gestore di trasporto pubblico locale, che vede l'impiego di un sistema AVM per una gestione più efficiente del traffico. L'infrastruttura, che abilita il monitoraggio in real-time della flotta di autobus, prevede una centrale operativa dotata di server ridondati, postazioni client, sistemi di deposito per lo scambio dati, e numerosi sistemi di bordo. La collocazione di queste risorse non viene resa nota, d'altro canto viene evidenziata la partnership con Vodafone, di conseguenza è lecito pensare che l'infrastruttura eretta da parte di Roma TPL possa essere, in prima battuta un centro di controllo che sfrutta server e altre apparecchiature acquistate e gestite internamente, seguendo un modello di deployment Private Cloud, posizionando l'hardware necessario, gestito dal reparto IT interno, localmente a una delle sedi della compagnia. In seconda battuta, l'architettura potrebbe sfruttare l'hardware messo a disposizione da un service

provider che offre risorse in maniera multi-tenant, seguendo un modello di deployment Public Cloud, e garantendo un modello di servizio Infrastructure as a Service (IaaS), oppure Platform as a Service (PaaS). Si noti che Vodafone è citato come terza parte nella soluzione, perciò la seconda opzione sopra citata è la più probabile, dato che uno dei servizi che offre uno dei colossi della telecomunicazione in Italia e non solo, tale Vodafone, è proprio quello di Public Cloud [VOD01]. Ad avvalorare ciò si cita Paul Greenley che afferma: “Around these group data centres, we have what we call satellite data centres: smaller facilities that still adhere to group standards but are run by local markets.” [DAT25], questo porta a pensare che, con alta probabilità, uno dei mercati locali che abilita i data center satellite sia proprio quello dei trasporti pubblici, di cui rappresentativo è il caso Roma TPL.

Le risorse necessarie alla pipeline di telemetria sviluppata possono essere messe a disposizione da entrambe le soluzioni presentate nel caso di studio appena descritto, si illustrano di seguito pregi e difetti di ognuna.

- **Private Cloud locale a una delle sedi della compagnia.** Modello di deployment che prevede la gestione delle risorse necessarie da parte di un'unica organizzazione che, normalmente, ne detiene la proprietà. Spesso questa soluzione è adottata da organizzazioni che predispongono l'infrastruttura per uso e consumo dei propri utenti, e dei dipartimenti interni, mentre un'alternativa è quella di demandare la gestione a una terza parte. Il modello in questione possiede le caratteristiche principali di offrire massime prestazioni e il massimo livello di sicurezza, dato che le risorse sono utilizzate per intero dal workload della compagnia. D'altra parte una soluzione di questo genere comporta costi elevati, dovuti alla necessità di acquisto dell'intero hardware, che normalmente comporta un investimento iniziale sostanzioso, e costi ricorrenti causati dalla configurazione e dalla manutenzione dell'hardware. Ne risulta di conseguenza una bassa scalabilità dell'infrastruttura, che risulta scalabile solamente in maniera verticale, ossia acquistando altre risorse.
- **Public Cloud.** Modello di deployment che prevede la gestione delle risorse necessarie da parte di un'organizzazione che offre i propri servizi a un numero elevato di utenti, dove la gestione dell'infrastruttura è completamente

a carico della compagnia che offre i servizi cloud. Il modello pubblico offre la massima scalabilità a prezzi minori, spesso personalizzabili, dove la scalabilità è garantita dall'enorme disponibilità di risorse da parte dell'organizzazione, in grado di mettere a disposizione dei propri utenti hardware in base alle esigenze del carico di lavoro del cliente stesso, permettendo scalabilità orizzontale elevata, mentre la gestione dei costi è Pay-per-Use, questo garantisce ai clienti del service provider una totale trasparenza, dato che i costi sono comportati dall'utilizzo delle risorse effettuato da parte del cliente. L'altro lato della medaglia di questa soluzione è rappresentato da prestazioni non elevate e sicurezza moderata, svantaggi dati dalla condivisione di risorse che avviene tra i diversi clienti della compagnia, motivo per il quale si hanno prestazioni inferiori alla soluzione Private Cloud, e dalla maggiore distanza che i dati devono percorrere per essere processati, che aumenta la superficie d'attacco esponendo i dati stessi a maggiori rischi.

Si noti che la prima possibilità implica che l'azienda che offre servizi di trasporto abbia disponibilità finanziarie sufficienti per l'acquisto, la configurazione, e la manutenzione dell'hardware necessario, che deve essere acquistato in quantità proporzionale alla flotta di mezzi da gestire. Ancora più importante l'azienda deve disporre di un team IT competente, e in grado di gestire un'infrastruttura Private Cloud a uso interno e dei clienti, ad esempio con strumenti come OpenStack, caratterizzato da una barriera d'ingresso importante. Supponendo di volere togliere dal carico dell'organizzazione questi aspetti, la seconda soluzione è quella che fa al caso nostro, predisponendo la possibilità di minori costi, dato che il modello è Pay-per-Use e non necessita di investimenti iniziali, e la necessità di un reparto IT interno con minori competenze, dato che il service provider si occupa di gestire l'hardware necessario. Quest'ultimo necessita di una gestione attraverso un modello di servizio che quanto meno è IaaS, ma che può essere PaaS nel momento in cui non siano disponibili le capacità di gestire l'ambiente di produzione, a lato del software che compone la pipeline di telemetria.

Entrambe le soluzioni hanno un fattore che le accomuna, ossia la necessità di connettività a Internet costante con alto throughput e bassa latenza, dato che il trattamento dei dati deve avvenire in real-time. Ecco perché per il sistema AVM

sviluppato si è pensato all'Edge Computing, modello che avvicina l'elaborazione dei dati alla fonte, fornendo la possibilità di un'elaborazione in real-time con migliore latenza e throughput, di garantire una migliore affidabilità in caso di limitata connettività, dato che parte dei dati sono processati in locale, di processare le informazioni maggiormente sensibili più vicino alla sorgente, limitando la superficie di attacco, e di diminuire la quantità di traffico che è necessario trasmettere verso il Public Cloud prima di avere una risposta sui dati monitorati, abilitando la possibilità di prendere decisioni strategiche con un numero maggiore di informazioni, spostando alcune analisi on-Edge.

1.2.2. Processing

Lo strato di processamento dell'architettura ha comportato la scelta tra batch processing e stream processing, con il primo maggiormente adatto a elaborare grandi quantità di dati con maggiore tolleranza sui tempi di risposta e garanzie di un throughput più prestante, mentre il secondo è pensato per un flusso di dati real-time con tempi di risposta molto rapidi e un'elaborazione basata su event-time o processing-time sulla base delle esigenze, garantendo una migliore latenza. Risulta quindi naturale per la pipeline di telemetria sviluppata la scelta dello stream processing, dato che il flusso di informazioni va analizzato in real-time, assieme a un'elaborazione all'event-time, abilitata dalla produzione alla sorgente dell'istante temporale di produzione dell'informazione.

Una possibile alternativa vagliata è quella del processamento a micro-batch, però data l'esigenza maggiore di latenza, e l'esigenza di processare in maniera immediata dati che possono comportare criticità per i mezzi, l'elaborazione a micro-batch è stata scartata.

Infine, si noti che è proprio il layer del processamento all'interno dell'architettura realizzata a risentire delle differenze nel trattamento di eventi, tant'è che questi ultimi vengono solamente memorizzati dallo Storage layer e mostrati dal livello di Visualizzazione, ritenendo non necessaria la loro elaborazione, al contrario della telemetria prodotta dai veicoli osservati.

Capitolo 2. Strumenti e tecnologie utilizzate

Strumenti e tecnologie inserite nel sistema AVM progettato e realizzato svolgono un ruolo ben preciso, a partire dalla comunicazione fino alla visualizzazione delle misurazioni raccolte, ogni tecnologia apporta funzionalità a un differente livello della soluzione. Verranno studiate le caratteristiche di ogni tecnologia, con particolare attenzione alle componenti fondamentali al funzionamento della pipeline di telemetria.

2.1. Protocollo di comunicazione - MQTT

La scelta è ricaduta su MQTT, per il modello push-based che permette di inviare messaggi e dati ai topic, e immediatamente i consumer iscritti risultano in grado di ricevere il messaggio, quindi semplifica la comunicazione in un contesto real-time, e inoltre diminuisce la latenza. Si è considerato il protocollo CoAP, con cui MQTT può essere messo a paragone, perché anch'esso è pensato per dispositivi o reti con capacità limitate, di conseguenza è particolarmente adatto a un contesto IoT che risulta essere constrained. Entrambi i protocolli possiedono la proprietà di un header molto ristretto, ciò permette di adottare un formato dati che può portare un maggiore overhead, dato che la dimensione del messaggio è pressoché data dal solo payload. Quello che ha sbilanciato il confronto tra i due protocolli verso MQTT, è il suo maggiore livello di maturità in termini di comunicazione publish-subscribe, modello che viene messo a disposizione anche da CoAP, attraverso CoAP Observer, ma con minori funzionalità.

Inoltre, la scelta di MQTT è comprovata dalla possibilità di utilizzare questo protocollo di comunicazione integrato al sistema di Ingestion Apache Kafka, in maniera complementare, attraverso l'utilizzo di una soluzione di bridging [MQT01]. Il pattern che permette l'utilizzo dei due strumenti, prevede l'utilizzo di MQTT all'edge, per la comunicazione tra device, e successivamente entra in gioco il bridge verso Kafka, che permette la raccolta di dati da numerose fonti. La soluzione in questione, introduce un livello aggiuntivo alla classica architettura di pipeline di analisi dati che comprende Ingestion, Processing, Storage e Visualizzazione, ossia il livello di integrazione o Integration layer, che comprende proprio la funzionalità di bridging verso il livello di Ingestion.

Il protocollo MQTT dispone diverse versioni, la 3.1.1 è semplice, leggera, basata su broker, segue il modello publish/subscribe, ed è l'ideale per scenari vincolati in termini di risorse o di rete; la versione 5.0 rappresenta un'evoluzione della precedente estendendola largamente. Si presentano di seguito alcune delle caratteristiche della versione 3.1.1.

- **Modello publish/subscribe.** La comunicazione prevede tre attori principali, il primo è il publisher, quest'ultimo svolge il ruolo di colui che prepara e pubblica i messaggi sui topic, topic gestiti dal secondo attore, rappresentato dal broker. Il broker è il componente che permette di distribuire i messaggi ricevuti dai publisher a uno o più receiver, terzo attore sulla scena, in particolare a tutti quei receiver che hanno precedentemente espresso interesse, attraverso la subscription, al topic nel quale è stato pubblicato il messaggio. Questo particolare modello di comunicazione è detto push-based, tant'è che permette di ricevere in maniera istantanea il messaggio, senza la necessità di trasmettere, in un momento precedente, una richiesta esplicita.
- **Overhead di trasporto ridotto.** Permette di formare un header del pacchetto da trasmettere di dimensioni ridotte, in modo che in scenari vincolati dove il throughput è limitato, l'header del messaggio da trasmettere non occupi maggiore spazio del payload, in caso il corpo del messaggio sia costituito da un piccolo numero di dati.
- **Facilità di utilizzo.** La presenza di pochi comandi, quali connect, disconnect, publish, e subscribe, evita inutili complessità, al contrario introdotte in altri protocolli.
- **Semantiche di consegna dei messaggi.** Presenti tre differenti Quality of Service (QoS), termine utilizzate in maniera non del tutto appropriata, in grado di garantire un livello crescente di affidabilità, corrispondente d'altra parte a un maggiore costo. La QoS 0 è associata alla semantica at most once, dove è garantita la consegna del messaggio al più una volta, ma contemporaneamente non è assicurata la consegna. La QoS 1 è associata alla semantica at least once, ciò significa che viene assicurata la consegna, ma non è affatto garantito che il messaggio non venga consegnato più di una volta. La QoS 2 è invece associata alla semantica exactly once, il che vuol

dire consegna assicurata esattamente una volta, se però per la precedente semantica è sufficiente utilizzare il meccanismo dell'acknowledgement, qui è necessario un handshake, che implica un maggiore traffico di rete e maggiore overhead per ogni singolo messaggio, motivo per il quale nella soluzione adottata si è preferito l'utilizzo di QoS 1.

- **Connessioni persistenti.** Significa che i subscriber, che possono agire sia da publisher che da soli receiver, possono effettuare una disconnessione e le subscription effettuate vengono mantenute attive per loro dal broker, di conseguenza un messaggio pubblicato su un topic che fa parte delle subscription viene salvato per la futura lettura, che in particolare avverrà a seguito di una nuova connessione.

A lato di queste caratteristiche, in ogni caso mantenute, la versione 5.0 (quella adottata) ne introduce di nuove, tra queste si è utilizzata la gestione degli errori, permessa dai "reason code" forniti dal broker; broker che viene implementato da molte piattaforme, tra queste si è scelto di utilizzare il broker MQTT offerto da RabbitMQ, che dispone supporto a tutte le versioni del protocollo analizzato via specifico plugin.

Il topic tree sviluppato nella pipeline di telemetria realizzata segue un approccio gerarchico, come mostrato in Figura 2.1, concedendo la possibilità di ampliamento futuro dell'architettura. Il livello L0 costituisce la radice, i cui figli (livello L1) rappresentano le funzionalità coperte nella soluzione. Il livello L2 mantiene aperta la porta a uno scenario in cui, aggiungere altre tipologie di veicoli da monitorare non costituisce un problema, infine il livello L3 raggruppa per tipologie le misurazioni osservate da parte del sistema AVM.

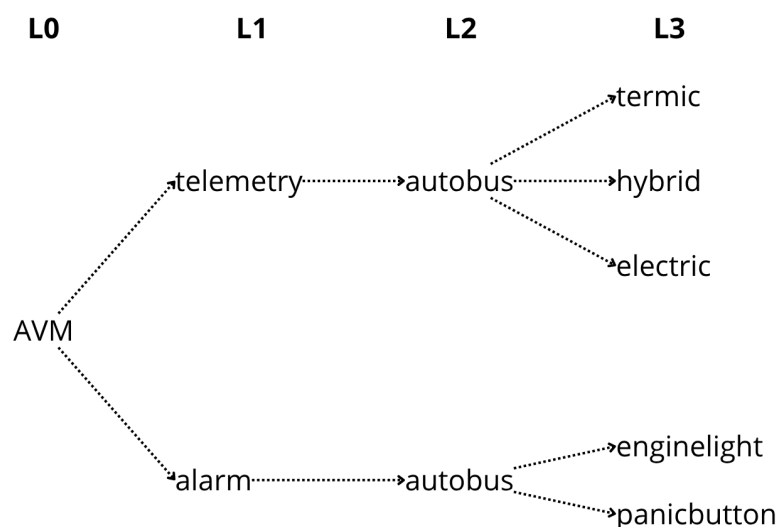


Figura 2.1. Immagine rappresentante il topic tree sviluppato per l'Integration layer.

2.2. Ingestion - Apache Kafka

La scelta condotta per il livello di Ingestion è rappresentata da Apache Kafka, selezionato per il successo avuto come piattaforma per la messaggistica e la comunicazione a qualsiasi livello, Enterprise e non, per le quali si propone come stato dell'arte. All'interno dell'architettura implementata possiede una collocazione on-Edge.

Kafka è un commit log, ossia tutti i messaggi vengono aggiunti in append a un file di log, distribuito, dato che esegue su un cluster, partizionato, quindi ogni topic è suddiviso in partizioni, e replicato, perché fa in modo che le partizioni siano replicate su più nodi del cluster. Il messaggio è l'unità fondamentale, tant'è che viene aggiunto in fondo al log in modalità persistente, ciò significa che fino alla scadenza della retention policy il messaggio persiste nel file di log, mentre scaduta la retention Kafka procede a eliminare il suddetto messaggio. La retention policy possiede diverse configurazioni, time-based, space-based e log compaction, che implica mantenere solo l'ultimo messaggio per ogni topic e scartare i restanti, nella soluzione realizzata si è percorsa la strada di una retention policy time-based. Il modello di comunicazione risulta essere pull-based, per cui i consumer leggono i dati sfruttando il puntatore, detto offset, che indica la posizione di lettura nel log, a

seguito di una richiesta di lettura esplicita. Si noti inoltre, che un messaggio ha una chiave, un valore e un timestamp, dove il valore è il contenuto del messaggio. I suddetti messaggi vengono categorizzati all'interno di topic, astrazione di log, che a loro volta sono suddivisi in partizioni, per cui i messaggi sono inseriti proprio in queste ultime, e Kafka è in grado di garantire ordinamento dei messaggi solamente all'interno di una partizione, non per l'intero topic.

Un singolo server Kafka è chiamato broker, le quali funzionalità vertono sulla ricezione dei messaggi da parte dei producer a cui assegna un offset prima di procedere alla scrittura sul topic, e sul recapito dei messaggi ai consumer a seguito di una richiesta di questi ultimi. I broker sono pensati per operare in cluster, dove eseguono come cluster controller; la proprietà di una singola partizione è di un solo broker del cluster, che assume il ruolo di partition leader, mentre altri broker, su cui la partizione è replicata, svolgono il ruolo di follower. Al fine di sfruttare queste e altre funzionalità si è scelto proprio l'utilizzo di un cluster Kafka, invece della più semplice possibilità stand-alone.

Il cluster Kafka può essere controllato attraverso due principali modalità, Kafka Raft (KRaft) e ZooKeeper, tra le quali si è scelta la prima: KRaft, quest'ultimo è il protocollo di consenso, introdotto per rimuovere la dipendenza di Kafka da ZooKeeper per la gestione dei metadati [CON01]. L'utilizzo di Kafka sotto forma di cluster è il fattore abilitante per sfruttare appieno alcune funzionalità:

- la prima funzionalità sfruttata ha luogo nel modo in cui sono consumati i messaggi, a svolgere questo compito sono i consumer, questi ultimi operano all'interno di un consumer group, costituito da uno o più consumatori che lavorano assieme per leggere i messaggi di un topic. Dato che il consumer group assicura che la lettura di una partizione avvenga da parte di un solo consumer, e che l'ordinamento dei messaggi viene garantito solamente all'interno di una partizione, il risultato è rappresentato da un notevole vantaggio di bilanciamento del carico, sfruttato nel momento in cui due o più consumer, parte dello stesso consumer group, eseguono contemporaneamente;
- la seconda funzionalità è data dalla replicazione su più nodi del cluster, altrimenti non sfruttata nella modalità stand-alone, e dalla definizione di

replica in salute. Tant'è vero che la replicazione permette di avere più copie dei dati all'interno del cluster, copie gestite dai broker, questo comporta una maggiore affidabilità. Secondo il grado di replicazione n un broker assumerà il ruolo di partition leader, mentre i restanti $n-1$ svolgeranno il ruolo di follower. A partire da ciò è possibile definire lo stato di salute di una replica, quest'ultima non è viva o morta, ma in sync, ciò significa che le repliche devono soddisfare le condizioni di avere una sessione attiva, e non rimanere troppo indietro rispetto al leader nella gestione della partizione [KAF25], assicurando che quest'ultima sia correttamente mantenuta da un numero minimo di broker, corrispondente al numero di repliche in sync. Il risvolto al lato pratico è dato dalla condizione secondo cui il numero di repliche in sync, anche dette In Sync Replicas (ISR), non deve mai superare il grado di replicazione.

Si è definito un topic tree anche per il layer di Ingestion, per fornire continuità rispetto alla struttura gerarchica definita in MQTT. In Apache Kafka però i topic risultano avere una struttura flat, per cui non può essere definita una vera e propria gerarchia, dato che i topic corrispondono a file di log all'interno del File System. La gerarchia sussiste solamente nella denominazione del nome file, in cui è possibile separare i livelli con un delimitatore, come il carattere “.” o “_”, solamente per maggiore chiarezza. Come mostrato in Figura 2.2, sono presenti livelli pseudo gerarchici del tutto simili all'albero definito per MQTT, con l'aggiunta di un livello, data la presenza di topic per il processamento, che consente di distinguere i differenti scopi dei dati inviati dal sistema di elaborazione.

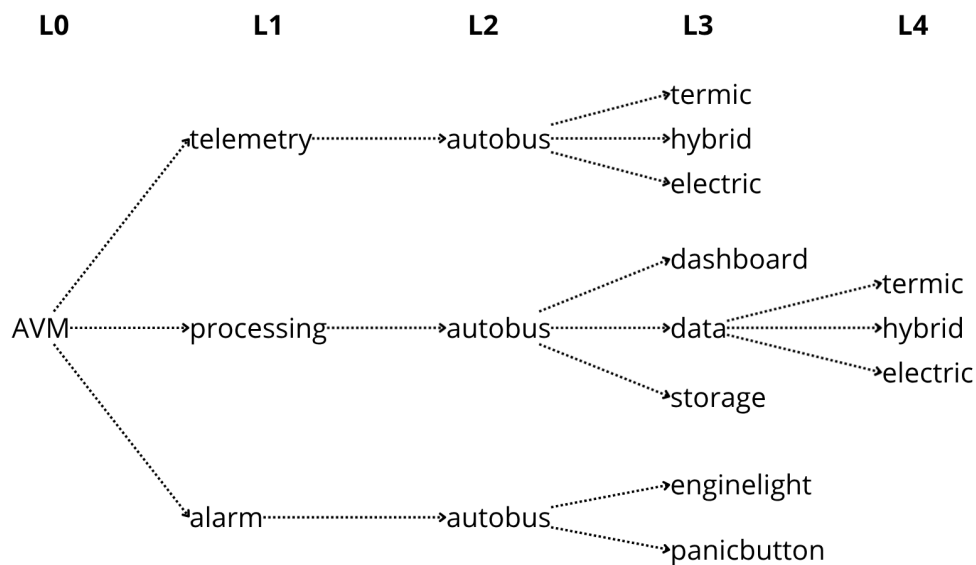


Figura 2.2. Immagine rappresentante lo pseudo topic tree realizzato per il layer di Ingestion.

2.3. Processing - Apache Flink

La scelta per lo strato di Processing è Apache Flink, uno tra gli strumenti maggiormente dominanti all'interno del campo del processing stream-oriented per il processamento in real-time dei dati. Tecnologia selezionata anche per il supporto fornito a una elaborazione all'event-time. La collocazione on-Edge del framework nel sistema AVM sviluppato permette un'analisi immediata.

Apache Flink è un framework di elaborazione distribuito, basato su Java, per calcoli stateful su flussi di dati bounded e unbounded. Nei primi i dati hanno inizio e fine, per cui è possibile effettuare una ingestion completa prima di eseguire una qualsiasi elaborazione, mentre nei secondi non è possibile eseguire una ingestion completa prima del processamento, perché non è prevista una fine al flusso di dati, ciò comporta la necessità di un processamento continuo [APA26]. Flink predispone diversi casi d'uso per il cui utilizzo è stato pensato, ossia applicazioni event-driven, applicazioni di data analytics, e applicazioni di data pipeline. Un'applicazione di stream processing è composta da alcuni blocchi fondamentali, presentati di seguito.

- **Streams.** Flink è in grado di processare sia bounded che unbounded streams, ma in aggiunta a questi anche real-time e recorded streams, dove il

primo implica un processamento nel momento in cui l'evento arriva al framework, mentre il secondo comporta la memorizzazione nel file system e un processamento a posteriori.

- **State.** Ogni applicazione non banale deve essere stateful, quindi necessita di mantenere uno stato per l'elaborazione. Flink offre primitive per diverse strutture dati, pluggable backend per mantenere lo stato, consistenza exactly-once, e inoltre è in grado di mantenere stati di grandi dimensioni, fino ad arrivare all'ordine dei TB.
- **Time.** La maggior parte delle elaborazioni sono basate sul tempo, per questo Flink offre diverse modalità di elaborazione a riguardo: event-time, watermark, late data handling, e processing-time.
- **Layered APIs.** Vi sono tre differenti API disponibili.
 - **Process Functions.** API maggiormente espressiva ma anche meno concisa, permette di processare singoli eventi da uno o più streams attraverso un controllo a grana fine.
 - **DataStream API.** Fornisce primitive per operazioni di elaborazione comuni, come il windowing.
 - **SQL & Table API.** Fornisce un'interfaccia in cui le query sono eseguite con la stessa semantica sia per stream unbounded che per stream bounded.

Lo strato di Processing della pipeline di telemetria realizzata è stato sviluppato come insieme di due casi d'uso, in particolare data pipeline e data analytics. Questi due predispongono un tipico utilizzo di alcune tipologie di blocchi fondamentali, in particolare per gli streams si fa largo uso di keyed streams, mentre per la fondameta del time, a rappresentare un ruolo decisivo sono watermarks, gestione di eventuali dati in ritardo, e finestre temporali.

I keyed streams si ottengono attraverso l'utilizzo del KeySelector keyBy(), con cui è possibile partizionare gli stream di dati su un certo attributo di ogni singola unità del flusso, in questo modo gli eventi che hanno lo stesso valore per quel determinato attributo sono raggruppati insieme. Le chiavi utilizzate nei KeySelectors possono

essere calcolate, e non solamente riferite come etichette, fornendo un meccanismo maggiormente complesso, ma allo stesso tempo dotato di notevoli potenzialità. Si noti inoltre la presenza di stato implicito, questo comporta un'analisi sulla quantità di stato necessario all'applicazione. Per evitare la crescita e il raggiungimento di grandi dimensioni, sono preferite aggregazioni su finestre finite e non direttamente riguardanti lo stream di dati unbounded.

Le finestre temporali costituiscono il meccanismo di Apache Flink per dividere data streams unbounded in frammenti finiti per aggregazione e analisi, in particolare sono un'astrazione che permette di raggruppare eventi secondo diversi criteri, al fine di eseguire operazioni come somma, media e altre ancora [CON26]. Il framework in analisi fornisce due categorie di finestre, le time-based e le count-based, dove le prime raggruppano eventi sulla base del timestamp, mentre le seconde sulla base del numero di elementi. Come tutti i framework di stream processing, Flink mette a disposizione l'elaborazione sulla base del processing time, ossia l'istante in cui l'evento entra nel framework, ma oltre a questo offre l'analisi secondo l'event-time, ossia l'istante in cui l'evento è stato prodotto, inserito all'interno dei dati stessi. Per tracciare il progresso dell'event-time Apache Flink utilizza il meccanismo del watermark, dove un watermark di tempo T indica che tutti gli eventi con timestamp inferiore o uguale a T sono presenti nel sistema. Tale meccanismo consente alle finestre temporali di valorizzare l'istante di fine finestra, e procedere a calcolare l'operazione definita sugli eventi che ricadono nell'intervallo di tempo corrispondente, inoltre permette di gestire eventuali ritardi, dato che se l'evento che entra nel sistema possiede un event-time minore del watermark, viene comunque compreso all'interno della finestra non ancora chiusa, nel caso in cui quest'ultimo sia in ritardo di un ammontare di tempo definito come tolleranza al momento della definizione della finestra. Apache Flink offre tre diverse tipologie di finestre temporali, presentate di seguito.

- **Tumbling Windows.** Gli eventi fanno parte di una sola finestra, dato che quest'ultima viene definita di dimensione fissata e non sovrapponibile. Le Tumbling Windows sono adatte a casi d'uso come report orari.
- **Sliding Windows.** Definita con parametri di dimensione e sliding fissati, le finestre della tipologia in analisi sono sovrapponibili se il parametro di sliding

definito risulta minore della dimensione della finestra stessa. Un caso d'uso rappresentativo è l'analisi di trend in real-time, tant'è che le Sliding Windows costituiscono la scelta di processamento dati nel sistema AVM realizzato.

- **Session Windows.** Raggruppano eventi sulla base di pattern di attività, perciò sono la scelta ottimale per casi d'uso come l'analisi di sessioni utente per un sito web.

Gli approcci per elaborare gli eventi all'interno delle finestre temporali Flink sono molteplici, `ReduceFunction`, `AggregateFunction`, e `ProcessWindowFunction`, l'ultima è la scelta per la pipeline di telemetria sviluppata, date le capacità di totale flessibilità e di totale accesso a dati e metadati di contesto fornite.

Per l'esecuzione del job Flink la scelta è ricaduta sull'esecuzione in application cluster. Un'applicazione Flink è una qualsiasi applicazione che lancia uno o più job Flink, l'esecuzione di questi job può avvenire su una JVM locale (`LocalEnvironment`) oppure su un cluster remoto (`RemoteEnvironment`), la prima soluzione individua un Flink Application Cluster, mentre la seconda un Flink Session Cluster, con differenze sul ciclo di vita e sull'isolamento delle risorse. Nel Flink Application Cluster non esiste nessun cluster già avviato, l'applicazione viene inviata all'eseguibile Flink, e lui stesso si occuperà dell'avvio di quest'ultima. Questo consente di avere applicazioni separate, e un ciclo di vita del cluster che è strettamente legato a quello dell'applicazione.

2.4 Storage e Visualizzazione - MongoDB e Grafana

Entrambi i layer finali della pipeline di telemetria realizzata sono stati collocati in Cloud, per lo Storage layer la scelta condotta è quella di MongoDB, dato che il formato dati è il JSON, risulta molto semplice pensare a un DataBase Management System (DBMS) Not only SQL (NoSQL) strutturato su un modello document store, di cui MongoDB è una delle soluzioni più affermate. Per il layer di Visualizzazione si è scelto lo strumento Grafana, che a livello di visualizzazione risulta essere sicuramente posizionato allo stato dell'arte.

MongoDB è un document store, dove il concetto centrale è il "documento". Ogni document store si basa sull'assunzione che i documenti incapsulino e memorizzino

dati o informazioni in alcuni formati o codifiche standard, come eXtensible Markup Language (XML), YAML Ain't Markup Language (YAML), JSON, e formati binari come Binary JSON (BSON), quest'ultimo rappresenta proprio il formato utilizzato da MongoDB. Una delle caratteristiche distintive dei document store è data dall'indicizzazione dei documenti nel database, quest'ultima avviene attraverso una chiave, unica nell'intero database, che rappresenta quel documento. MongoDB, come metodo di organizzazione e raggruppamento dei documenti, dispone le collezioni. Queste ultime, rispetto ai database relazionali, potrebbero essere considerate analoghe a tabelle e i documenti analoghi ai record, in realtà i due concetti sono profondamente diversi, tant'è vero che ogni record in una tabella ha la stessa sequenza di campi, mentre i documenti in una collezione possono avere strutture completamente differenti. Le collezioni in MongoDB possiedono due tipologie, di cui una è la "timeseries collection", quest'ultima comporta la specifica di alcuni parametri come il timestamp, i metadati, e la granularità dei dati (secondi, minuti, ore).

Essendo MongoDB posizionato in Cloud, la connessione all'istanza online può avvenire tramite browser, accedendo al proprio spazio in cloud e creando un cluster che è in grado di ospitare un limitato ammontare di dati, oppure attraverso Compass. MongoDB Compass è una potente interfaccia grafica per l'interrogazione, l'aggregazione, e l'analisi dei dati MongoDB in un ambiente visivo [MON01]. Questo applicativo permette la connessione attraverso un apposito URI, fornito direttamente dal cluster al momento della prima connessione.

Grafana è un'applicazione Web open source di analisi e visualizzazione interattiva, che consente di acquisire dati da un enorme numero di fonti, interrogarli e visualizzarli su grafici personalizzabili per una facile analisi. Inoltre è possibile impostare avvisi, detti alert, in modo da poter ricevere in maniera immediata una notifica riguardo comportamenti anomali. In sintesi, Grafana consente di trasformare l'ammasso di metriche, osservate e raccolte, in visualizzazioni. Tutto questo rende possibile prendere decisioni fondamentali al fine di mantenere integro lo stack di applicazioni [MED23].

Capitolo 3. Analisi del funzionamento

Il flusso di comunicazione all'interno dell'architettura è rappresentato dagli autobus smart, con diverse motorizzazioni, che inviano le metriche raccolte verso il sistema di Ingestion, Apache Kafka, mediante il protocollo di comunicazione MQTT. Successivamente le informazioni passano dal sistema di Ingestion verso quello di Processing, ossia Apache Flink, poi, una volta elaborati, i dati transitano al sistema di Storage, rappresentato da MongoDB, e infine al sistema di Visualization, Grafana (Figura 3.1). Qui l'architettura opera storage dei dati non solo dopo averli processati, ma prevede una memorizzazione anche dei raw data forniti dagli autobus allo strato di Ingestion, con un'ottica Medallion Architecture.

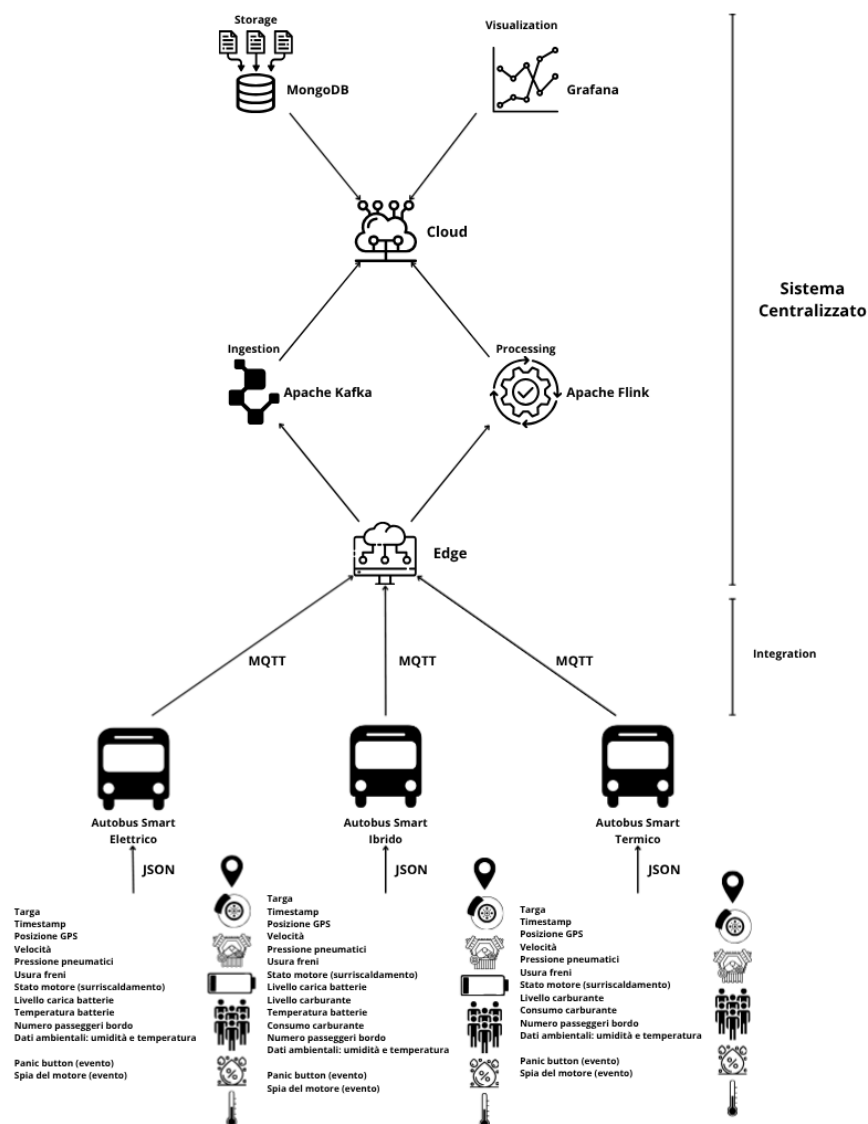


Figura 3.1. Immagine rappresentante l'architettura del sistema AVM realizzato.

3.1. Produzione dati

La produzione dati prevede la simulazione dei veicoli osservati dal sistema AVM sviluppato. Sfruttando la caratteristica OOP del linguaggio di programmazione Python, si è realizzata una gerarchia di classi per rappresentare gli autobus, in particolare la superclasse `Autobus`, e le sottoclassi `AutobusTermico`, `AutobusIbrido`, e `AutobusElettrico`, modellate sulla tipologia di motorizzazione dei mezzi di trasporto. Si sono sviluppati di conseguenza file diversi per una migliore separazione, ognuno contenente una classe. La classe padre è composta da tutto ciò che risulta essere comune e indipendente dalla tipologia di veicolo, mentre le classi figlio estendono la classe padre compiendo override di alcuni metodi, in particolare quelli in cui è necessaria l'aggiunta di funzionalità type-specific.

L'effettiva produzione dei dati implica la generazione casuale di questi ultimi in intervalli sensati per ogni metrica, motivo per il quale sono state condotte ricerche per i range in cui produrre alcune metriche, tra cui la temperatura del pacco batterie, le coordinate di latitudine e longitudine, e le dimensioni del serbatoio. Per quanto concerne la produzione di latitudine e longitudine, si è scelto di simulare un percorso unico di prova del sistema realizzato tra Ferrara e Bologna, inserendo quindi come range latitudine e longitudine delle due città. La temperatura del pacco batterie risulta problematica nel momento in cui si superano i 45 °C, oppure nel momento in cui la temperatura dovesse scendere al di sotto dei 15 °C [ELE23], per questo come range di produzione metrica è stato selezionato [5, 55] °C, in modo da poter produrre valori sia al di sotto dei 15 °C, che al di sopra dei 45 °C, al fine di simulare una problematica grave. Le dimensioni del serbatoio variano sulla base del tipo di bus, urbano o extraurbano, dell'autonomia e dell'alimentazione. Nonostante questo una dimensione adatta a tratti extraurbani, è data da un serbatoio in grado di contenere tra i 400 e i 600 litri di carburante [VOL01].

Il vero e proprio ciclo di funzionamento è formato da quattro azioni principali, la simulazione delle metriche, la preparazione del pacchetto dati, la comunicazione dei dati di telemetria, e la stampa a video delle informazioni trasmesse. La simulazione delle metriche avviene secondo quattro diversi algoritmi, il primo, dedicato alla simulazione di latitudine e longitudine, prevede l'aggiornamento dei dati aggiungendo o sottraendo una quantità prestabilita al fine di coprire l'unico percorso

di prova su cui il sistema AVM è funzionante. Il secondo, dedicato alla maggior parte delle metriche, prevede uno scostamento massimo all'interno di un intervallo, per cui viene generata la metrica in maniera casuale, dopo di che se questa risulta all'interno dell'intervallo prestabilito viene mantenuta, altrimenti viene generata di nuovo all'interno dell'intervallo opportunamente modificato, questo per evitare aggiornamenti di metriche troppo repentini, ad esempio per evitare che in un istante la velocità riportata sia di 50 km/h, e dopo 5 secondi aumenti a 100 km/h, non avrebbe alcun senso, soprattutto per un mezzo pesante quale è l'autobus. Il terzo, dedicato alle metriche riguardanti i livelli rimanenti di carburante o di carica della batteria, prevede una semplice diminuzione di una quantità prestabilita fino a una determinata soglia, decisa casualmente, dopo la quale viene simulato il rifornimento riportando le metriche al valore massimo. Il quarto, dedicato alle metriche di consumo, prevede l'aggiunta della stessa quantità di carburante consumata.

La preparazione del pacchetto dati fornisce una metodologia per convertire i dati, prodotti fino a quel momento, in un formato standard, in particolare il JSON, formato di invio metriche selezionato per la pipeline di telemetria realizzata.

La comunicazione del pacchetto dati di telemetria (Listato 3.1) prevede l'acquisizione del client MQTT, precedentemente istanziato, del payload del pacchetto e del topic MQTT verso cui inviare il messaggio, qualora non vengano forniti come argomento. Successivamente avviene la verifica della connessione al broker MQTT, e solamente se quest'ultima è correttamente stabilita si procede all'invio del pacchetto dati a QoS 1. Nel momento in cui la connessione verso il broker MQTT dovesse essere assente, il metodo descritto procede alla memorizzazione della coppia topic - pacchetto all'interno di una coda, nell'attesa che la connessione sia ristabilita. Per questo viene utilizzato il metodo "loop_start" di Paho, API che fornisce gli oggetti necessari per la corretta gestione di un client MQTT in Python, che implementa un meccanismo automatico di tentativi infiniti di riconnessione nel momento in cui quest'ultima dovesse venire meno. Quando la connessione viene ristabilita, il metodo in analisi procede all'invio dei messaggi precedentemente posti in coda. Si noti inoltre la presenza di un flag che permette di distinguere il caso di comunicazione di un normale dato di telemetria, dal caso in cui è necessario comunicare un allarme.

```

# Comunicazione - comunicazione verso il sistema di Ingestion,
# ovverosia trasmissione del "pacchetto" dati verso
# il broker MQTT, e successiva predisposizione di un bridge per la
# comunicazione al sistema di Ingestion, ossia Kafka
def communicate(self, alarm=False, topic="AVM/info", message="hello"):
    mqtt_client = self.get_mqtt_client()
    mqtt_timeout = self.get_timeout()
    payload = self.get_formatted_data_to_send()
    topic_to_publish = self.get_topic()
    msg_queue = self.get_msg_queue()

    # Verifica messaggio di allarme da comunicare e
    # conseguente assegnazione di messaggio e topic passati
    # come argomenti di funzione
    if alarm:
        topic_to_publish = topic
        payload = message

    # Verifica connessione client to broker
    if mqtt_client.is_connected():
        # ... Invio messaggi in coda ...

        # Cancellazione dalla coda di tutti i messaggi
        # precedentemente in attesa
        msg_queue.clear()
        self.clear_msg_queue()

        # Publish con QoS 1 per assicurare la consegna del
        # messaggio corrente
        msginfo = mqtt_client.publish(topic=topic_to_publish,
payload=payload, qos=1)

        # Attesa della pubblicazione del messaggio per
        # assicurare una corretta gestione della QoS
        # desiderata. QoS = 1 indica una qualità del servizio
        # 'at_least_once'
        before_wait = time.time()
        msginfo.wait_for_publish(timeout=mqtt_timeout)
        after_wait = time.time()
        if after_wait - before_wait >= mqtt_timeout:
            print(f"Uscita da wait_for_publish() --> timeout di
{mqtt_timeout} s scaduto")
        else:
            print(f"Uscita da wait_for_publish() con successo della
pubblicazione sul broker del messaggio corrente")
        else:

```

```
# Aggiunta (topic, messaggio) non inviato alla coda di
# messaggi in attesa
self.append_msg_queue(el=(topic_to_publish, payload))
print("Connessione assente --> messaggio in coda...")
```

Listato 3.1. Frammento di codice rappresentante il metodo del veicolo utile alla comunicazione dei dati.

Integrata all'interno del ciclo di funzionamento è presente la gestione eventi. Questi ultimi vengono preparati specificando il tipo dell'evento, la targa dell'autobus a cui è associato l'evento, l'istante di creazione, e le informazioni associate al veicolo di posizione e numero di passeggeri a bordo. Successivamente viene generato un numero da specifiche distribuzioni gaussiane, descritte nella sezione 1.1.3, e verificata la relazione che quest'ultimo possiede con il numero uno, solamente nel momento in cui il numero estratto dalla distribuzione risulta essere maggiore di uno, l'evento accade. L'avvenimento dell'evento comporta la gestione da parte dell'opportuno autobus, l'aggiornamento di liste e altri oggetti contenenti il riferimento a quel determinato autobus, e la generazione di un nuovo evento. La gestione da parte del veicolo è composta da due azioni, l'invio dell'allarme e la fermata definitiva dell'autobus. L'invio dell'allarme avviene attraverso l'utilizzo dello stesso metodo utile alla comunicazione della telemetria, ma con il passaggio di opportuni argomenti, quali topic, messaggio, e flag di allarme. La fermata definitiva dell'autobus avviene attraverso il metodo "stop_autobus", quest'ultimo prevede la disconnessione dal broker e la fermata del loop di servizio MQTT. Si noti, infine, che il metodo "stop_autobus" è lo stesso utilizzato dal sistema quando perviene un segnale SIGINT, al fine di garantire una corretta terminazione.

3.2. Integration layer

L'Integration layer è lo strato aggiuntivo previsto dal sistema AVM realizzato al fine di istituire un ponte tra il traffico MQTT e il traffico Kafka, ossia recapitare tutto ciò che il broker MQTT riceve al cluster Apache Kafka. A questo scopo si sono definite le connessioni ai due broker, con una particolarità per Kafka, ossia l'utilizzo dell'oggetto KafkaAdmin, messo a disposizione da parte della libreria kafka-python. Quest'ultimo permette lo svolgimento di alcune azioni con ruolo amministratore, tra le quali la creazione dei topic, proprio quest'ultima è l'obiettivo gestito manualmente, dato che

la creazione automatica del topic prevede parametri di partizionamento, replicazione e ISR minime impostate all'unità. Nella soluzione realizzata si è optato per l'utilizzo di due partizioni per ogni topic, fattore di replicazione a tre, dato che tre sono i broker presenti nel cluster gestito, e tre è anche il numero di ISR minime predisposto.

Il modello di programmazione per la gestione del client MQTT offerto da Paho è asincrono, per questa motivazione si sono implementate una serie di callback, funzioni invocate al momento dell'avvenimento di uno specifico evento. La callback di maggiore rilevanza è sicuramente "on_message", quest'ultima permette di definire il comportamento del client al momento della ricezione di un messaggio dai topic verso cui ha espresso interesse attraverso le subscription. L'algoritmo sviluppato per la callback di ricezione del messaggio prevede la gestione di eventuali duplicati. Tant'è vero che tra i diversi parametri contenuti nel messaggio ricevuto, Paho inserisce il flag DUP, che permette di discriminare tra un messaggio originale e un messaggio duplicato. Questa funzionalità risulta particolarmente importante dato l'utilizzo di una QoS 1, ossia di una semantica di consegna at least once. Il DUP flag viene valorizzato a "True" solamente nel momento in cui il messaggio viene comunicato di nuovo dal publisher, a seguito della mancata ricezione dell'acknowledgement da parte di quest'ultimo, che però non comporta il non aver ricevuto e processato il messaggio da parte del consumer. Per questa motivazione si è predisposto un dizionario Python che mantiene in memoria gli ultimi 120 messaggi ricevuti, o meglio, per ogni messaggio mantiene l'identificativo come chiave e come valore il timestamp inserito nel payload. Questo meccanismo garantisce una limitata copertura da casi di riprocessamento dello stesso messaggio, tant'è che se nel sistema sono presenti 30 autobus, ossia il numero massimo di veicoli monitorati dalla pipeline di telemetria, sono mantenuti i messaggi dell'intervallo di tempo $[t - 20, t]$ secondi, con t istante di tempo corrente; mentre al contrario con il minimo numero di autobus per tipologia di motorizzazione, ossia 3 autobus in totale, sono memorizzate le informazioni dei messaggi dell'intervallo di tempo $[t - 120, t]$ secondi, con t istante di tempo corrente. Quindi per un intervallo di tempo dipendente dal numero di mezzi di trasporto in marcia, l'Integration layer è in grado di inferire sul momento in cui è accaduto il processamento di un determinato messaggio che entra nel sistema. A seguire l'algoritmo implementa l'effettiva comunicazione del messaggio da MQTT ad Apache Kafka, passando per la

trasformazione del topic MQTT in un topic Kafka, sostituendo il delimitatore del primo, ossia “/”, con il delimitatore “.”, dato che i topic Kafka vengono mappati in log del File System.

3.3. Ingestion & Storage layers

Lo strato di Ingestion è stato posizionato on-Edge, per permettere una comunicazione a minore latenza e un maggiore controllo sui dati da parte del sistema AVM. La tecnologia di messaggistica utilizzata è Apache Kafka, il cui cluster viene contattato da parte del layer di Ingestion attraverso l’opportuna connessione predisposta. Come nell’Integration layer, anche lo strato in questione fa utilizzo dell’oggetto KafkaAdmin, nella stessa maniera descritta nella sezione 3.2.

Il modello di programmazione definito da Kafka fa sì che il consumatore debba inviare una richiesta al cluster per poter consumare un messaggio, per questa motivazione si definisce un ciclo infinito che continuamente fa polling al cluster in cerca di nuovi messaggi da consumare. Lo scopo principale del layer di Ingestion è raccogliere tutti i dati che le fonti inseriscono nei topic Kafka, tant’è vero che il consumatore in esecuzione definisce subscription a tutti i topic del sistema AVM, ossia i topic dedicati alla raccolta dei dati di telemetria osservati, i topic dedicati alla comunicazione di dati processati, e i topic dedicati all’invio di un qualsiasi tipo di allarme. Si rende quindi chiara l’adozione di un modello che fa riferimento alla Medallion Architecture, dato che lo strato di Ingestion contatta il layer di Storage per memorizzare non solo i dati processati, ma anche i dati di telemetria e gli allarmi, informazioni che restano grezze.

La principale funzionalità di questo livello dell’architettura è contattare lo strato di Storage, collocato in Cloud, per la memorizzazione di tutti i dati del sistema sviluppato. Questo avviene attraverso l’API Python messa a disposizione da MongoDB, di cui si è seguito il modello di programmazione sincrono, che permette di creare la connessione verso la propria istanza Cloud, creare o ottenere le collection necessarie, e infine memorizzare i documenti rappresentanti le informazioni ricevute e consumate da parte del KafkaConsumer. Quest’ultimo predispone i nomi delle collezioni MongoDB, e successivamente invoca il metodo “_store” che riceve in ingresso il nome della collezione e il documento da inserire. Il suddetto metodo

procede con l'ottenimento della lista di collezioni presenti nell'istanza Cloud MongoDB, col fine di verificare la presenza o meno della collezione fornita come argomento. Dato che per sfruttare il vantaggio di load balancing, conseguente all'esecuzione simultanea di più consumatori Kafka, si eseguono più processi riguardanti questo strato, l'operazione di verifica della presenza della collezione tra le collection istanziate in Cloud può generare una race condition. Quest'ultima può verificarsi se a seguito di una richiesta del listato delle collezioni presenti all'interno del cluster online da parte di un processo, quest'ultimo viene descheduled, a questo punto lo scheduler decide di assegnare la CPU a un secondo processo che procede con la medesima richiesta, verifica la presenza della stessa collezione del precedente processo, e crea la collezione se non presente, successivamente il processo in questione cessa temporaneamente l'esecuzione a causa dello scheduler. Alla nuova assegnazione della CPU al primo processo, l'istanza del programma procede con la creazione della collezione e cade in errore. Al fine di risolvere questo problema si è predisposta la cattura dello specifico errore verificatosi a seguito del tentativo di creazione di una collezione già presente (Listato 3.2), con un'azione correttiva che prevede il solo ottenimento della collezione.

```
# Reperimento db
database = self.get_mongodb_client().
.get_database(name=self.get_mongodb_db_name())

# Reperimento lista di collection presenti nel db
collection_list = database.list_collection_names()

# ...

# Verifica della presenza della collection d'interesse all'interno
# del db, nel caso in cui sia presente viene solamente reperita,
# mentre se non presente viene creata come timeseries collection,
# con annessa specifica di campo del timestamp della timeseries,
# campo di metadati della timeseries e granularità di
# quest'ultima. Per la granularità si è inserita quella di più
# basso livello possibile
ts_collection = None
if collection_name not in collection_list:
    # Creazione della collection inserita all'interno di un blocco
    # try-except perché nel momento in cui vengono eseguiti più processi
    # potrebbe scatenare una race condition, ad esempio un processo sta
```

```

# per creare la collection ma viene descheduled subito prima, dopo
# di ché viene schedulato un altro processo che crea
# esattamente la stessa collection, quando viene rischeduled il
# primo processo la creazione della collection fallisce perché
esiste
# già --> inserimento nel blocco try-except catturando le
# eccezioni corrispondenti a 'collection already exists'
try:
    ts_collection = database.create_collection(name=collection_name,
        timeseries={"timeField": "timestamp", "metaField":
            "metadata", "granularity": "seconds"})
except CollectionInvalid:
    print("Collection already exists --> get_collection")
    ts_collection = database.get_collection(name=collection_name)
except OperationFailure as e:
    if getattr(e, "code", None) == 48:
        print("Collection already exists --> get_collection")
        ts_collection =
database.get_collection(name=collection_name)
    else:
        raise
else:
    ts_collection = database.get_collection(name=collection_name)

```

Listato 3.2. Frammento di codice rappresentante le azioni opportunamente predisposte per rimediare a una possibile race condition.

Infine, il metodo procede con la manipolazione del documento ricevuto al fine di adattare quest'ultimo alla struttura richiesta da una "timeseries collection" MongoDB. Per cui nella totalità dei documenti si prevede la formazione dei metadata, mentre solamente per i documenti caratterizzati da dati processati, si predispone una maggiore aggregazione delle statistiche prodotte, soprattutto al fine di favorire la fruizione da parte dello strato di Visualizzazione.

3.4. Processing layer

Nella pipeline di telemetria realizzata si è collocato il Processing layer vicino alla fonte dei dati, ossia sull'Edge. La tecnologia di stream-processing selezionata è Apache Flink, per le motivazioni illustrate nella sezione 2.3. Il compito di questo strato prevede la ricezione dei dati di telemetria da processare, elaborare questi ultimi, e successivamente comunicare i risultati dell'analisi verso l'Ingestion layer,

oltre che presentare gli stessi in un'apposita dashboard direttamente prodotta vicino alla fonte. Per cui ancor prima dell'arrivo dei dati allo strato di Visualizzazione, posto in Cloud, è possibile avere un quadro generale dell'andamento dei veicoli in marcia, al fine di prendere decisioni informate nel più breve tempo possibile.

Lo strato in analisi prevede la connessione al cluster Kafka, attraverso l'oggetto `KafkaAdmin`, utilizzato come descritto nella sezione 3.2, e la definizione dell'ambiente Flink in cui eseguire il job, possibile attraverso l'uso dell'API Python messa a disposizione da Flink, ossia `pyflink`. Al fine di soddisfare la ricezione dei dati di telemetria, si è predisposto un `KafkaConsumer`, in grado di ricevere la telemetria comunicata dallo strato di Integrazione, incapsulato in una `KafkaSource`, oggetto fornito da `pyflink` che permette di definire una sorgente Kafka, con relativo topic, da cui ricevere dati. Allo stesso modo, dato che la piattaforma di messaging per l'intero sistema AVM è Kafka, si sono definiti i `KafkaSink` necessari. I sink sono in grado di inviare dati provenienti da stream bounded o unbounded al di sopra di topic Kafka. Per evitare problemi relativi a partizioni Kafka idle, si è impostato il parallelismo dell'ambiente Flink all'unità, e di conseguenza il partizionamento dei topic da cui il nodo di elaborazione riceve dati di telemetria all'unità. Questo vale anche per il topic Kafka dedito alla comunicazione dei risultati dell'elaborazione verso la dashboard on-Edge, con il partizionamento definito a uno che possiede un'altra valenza, ossia permettere al singolo nodo di processamento delle informazioni, e al singolo processo dedito alla presentazione della dashboard, di ricevere tutti i dati in maniera ordinata, dato che l'ordinamento in Kafka è garantito solamente sulla singola partizione. Al contrario, si è definito il partizionamento del topic dedito alla ricezione delle analisi da parte dell'Ingestion layer a due, al fine di distribuire il carico dei molti dati processati sui molteplici processi in esecuzione per il tale strato.

L'elaborazione dei dati passa dalla definizione del watermark, dall'estrazione del timestamp dai dati stessi, e dalla predisposizione della finestra temporale in cui elaborare le informazioni (Listato 3.3). Si è prevista una strategia di watermark che ammetta dati in ritardo per un massimo di 15 secondi, in grado di ignorare partizioni idle fino a 120 secondi, e dotata di assegnazione del timestamp proveniente dai dati. Successivamente è stato definito lo stream di dati dalla sorgente Kafka, specificando l'applicazione di nessuna strategia di watermark, dato che quest'ultima viene associata al data stream in un momento successivo. A questo proposito si

predispone il mapping dello stream precedentemente definito, allo scopo di applicare la funzione “parse_event” a ogni evento contenuto da tale stream unbounded, quest’ultima prevede la creazione di una nuova coppia chiave - valore all’interno del dato, il cui valore è il timestamp, con il fine di assegnare la strategia di watermark proprio a questo stream di dati trasformato, dato che la strategia prevede l’utilizzo della classe custom MyTimestampAssigner, che estrae l’event-time proprio dalla coppia chiave - valore aggiunta dalla funzione di parsing. Al fine di limitare il flusso di dati unbounded per l’elaborazione, si prevede la trasformazione dello stream in un keyed stream, raggruppando i dati secondo la targa, per applicare una finestra temporale di tipo Sliding Window, che comprende gli eventi riguardanti gli ultimi 60 secondi, con sliding di 5 secondi, in questo modo si verifica overlapping delle finestre, in particolare in un minuto si avranno ben dodici finestre temporali sovrapposte. Sui data stream, così limitati, viene successivamente applicata la funzione di elaborazione custom, definita come sotto classe di ProcessWindowFunction. Si noti, inoltre, che il processamento risulta in ritardo di 20 secondi durante l’esecuzione, questo motivato dall’attesa per l’avanzamento dell’event-time comportata dai 15 secondi di transitorio per eventuali dati in ritardo, in aggiunta a questo il watermark non avanza fino a quando non è disponibile nel sistema un nuovo evento dotato di timestamp più recente, e dato che le metriche sono prodotte ogni 5 secondi, sono proprio questi a comportare i 15 + 5 secondi di ritardo durante l’esecuzione.

```
# --> for_bounded_out_of_orderness() consente di ammettere
#     messaggi out of order e attendere questi fino a un tempo
#     massimo passato come argomento, inoltre per fare in modo che
#     questo avvenga il watermark è sempre "indietro" rispetto al
#     tempo corrente di questa quantità
# --> with_idleness() consente di ignorare partizioni idle del
#     topic Kafka (per non tenere bloccato il watermark globale)
watermark_strategy =
WatermarkStrategy.for_bounded_out_of_orderness(Duration.of_seconds(15))
\
.with_idleness(Duration.of_seconds(120)) \
.with_timestamp_assigner(MyTimestampAssigner())

# Creazione data stream a partire dalla source Kafka definita in
# precedenza, non specifica nessuna strategia per l'assegnazione
# dei watermark perché specificata in seguito associata a ogni
```

```

# messaggio proveniente dal data stream
ds = env.from_source(
    source=kafka_source,
    watermark_strategy=WatermarkStrategy.no_watermarks(),
    source_name="kafka_source_for_processing"
)

# Parse delle stringhe JSON a dizionari Python e assegnamento
# watermarks basati sul timestamp del dato di telemetria
ds_parsed = (
    ds
    .map(parse_event)
    .assign_timestamps_and_watermarks(watermark_strategy)
)

# key by license_plate, sliding window dimensionata a 60s, slide
# 5s, process per window
ds_windowed_processed = (
    ds_parsed
    .key_by(lambda d: d['license_plate'], key_type=Types.STRING()) \
    .window(SlidingEventTimeWindows.of(Time.seconds(60),
        Time.seconds(5))) \
    .allowed_lateness(time_ms=90000) \
    .process(MyProcessWindowFunction(tp_thres=tp_thres,
        bs_thres=bs_thres, es_thres=es_thres, bt_thres=bt_thres))
)

```

Listato 3.3. Frammento di codice rappresentante le azioni preparatorie di supporto all'elaborazione dei dati.

Il processamento viene effettuato sulla pressione delle gomme, sullo stato del motore, sullo stato dell'impianto frenante, e, per i veicoli con opportuna tipologia di motorizzazione, sulla temperatura del pacco batterie. In particolare si prevede il calcolo della media per pressione e temperatura, mentre il conteggio delle condizioni gravi per le rimanenti metriche. Successivamente si verifica la media della pressione delle gomme, e nel momento in cui quest'ultima risulta al di sotto della soglia prevista, ossia 2.0 bar, viene attivato l'allarme di condizioni critiche relativo. La verifica effettuata per la media della temperatura del pacco batterie è del tutto simile, se non per la relazione e la soglia, specifiche per la metrica. In particolare si controlla che il risultato dell'analisi superi la soglia di 45.0 °C, e in questo caso si prevede l'attivazione dell'allarme di temperatura. L'elaborazione che riguarda le rimanenti metriche è simile ma effettuata su un dato non mediato, tant'è che si

verifica se il numero di stati critici di motore e impianto frenante, risulta essere superiore a una specifica soglia negli ultimi 60 secondi, in caso affermativo si attiva l'allarme per la metrica associata.

Infine i risultati delle analisi condotte per ogni finestra temporale vengono inviati allo strato di Ingestion e alla dashboard on-Edge. Quest'ultima prevede la predisposizione di una tabella in cui visualizzare le informazioni ricevute dal nodo di elaborazione, in particolare si mostrano inizio e fine della finestra temporale, i risultati numerici, ossia medie e conteggi, e gli allarmi relativi a ogni metrica processata. Quest'ultima è uno strumento utile alla centrale operativa al fine di mantenere controllato lo stato giornaliero degli autobus smart, dato che il sistema di dashboarding mostra le informazioni essenziali di tutti e soli i veicoli in marcia nell'istante di tempo corrente.

3.5. Visualization layer

Il Visualization layer, come lo strato di Storage, è collocato in Cloud. Lo strumento selezionato è Grafana, che mette a disposizione funzionalità quali la creazione di dashboard, la definizione di alert, e l'utilizzo di una vasta quantità di fonti di dati da cui attingere. Le dashboard prodotte sono due, la prima riguarda le statistiche della telemetria osservata e la presentazione della telemetria stessa in una scheda, mentre le statistiche e i dati associati agli allarmi di spia motore e panic button in una seconda scheda. La seconda dashboard riguarda la presentazione dei dati processati e dei relativi allarmi, anch'essi organizzati in due schede differenti.

La dashboard di telemetria mostra, per ogni tipologia di motorizzazione, statistiche quali il numero di autobus, e minimo e massimo di tutte e sole le metriche numeriche, esclusi i consumi. Successivamente presenta i dati di telemetria, anch'essi divisi per tipologia di veicolo, nella loro totalità. Si riporta in Figura 3.2a e in Figura 3.2b proprio il frammento di dashboard rappresentante la telemetria per la tipologia di motorizzazione termica. Quanto descritto finora riguarda la scheda di telemetria, mentre la scheda dedicata agli allarmi di spia motore e panic button permette di visualizzare il numero di allarmi verificatisi, e inoltre i dati associati all'ultimo allarme ricevuto dal sistema, oltre a uno sguardo al numero e alle informazioni relative agli eventi critici (Figura 3.3).

La dashboard relativa ai dati processati mostra un report che consente di analizzare eventuali problemi di ogni autobus presente nel sistema, nella scheda che raccoglie le statistiche, mentre la scheda riguardante gli allarmi permette uno sguardo dettagliato alla situazione associata a ogni autobus, mostrando in aggiunta per quanto tempo perdura uno specifico tipo di allarme per un determinato autobus smart osservato dal sistema AVMM.

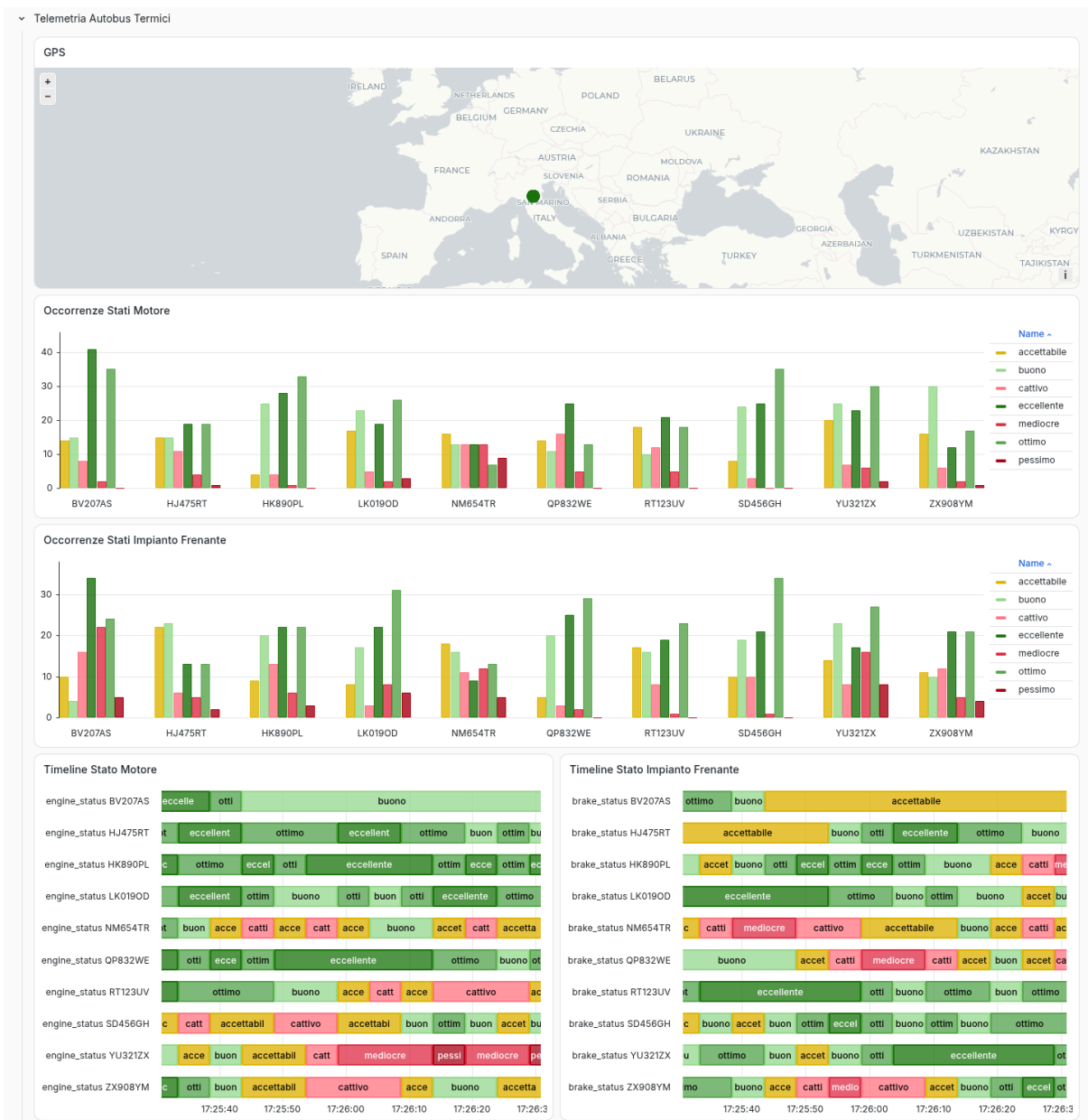


Figura 3.2a. Immagine rappresentante le statistiche dei dati di telemetria per autobus di motorizzazione termica.



Figura 3.2b. Immagine rappresentante le serie temporali dei dati di telemetria per autobus di motorizzazione termica.

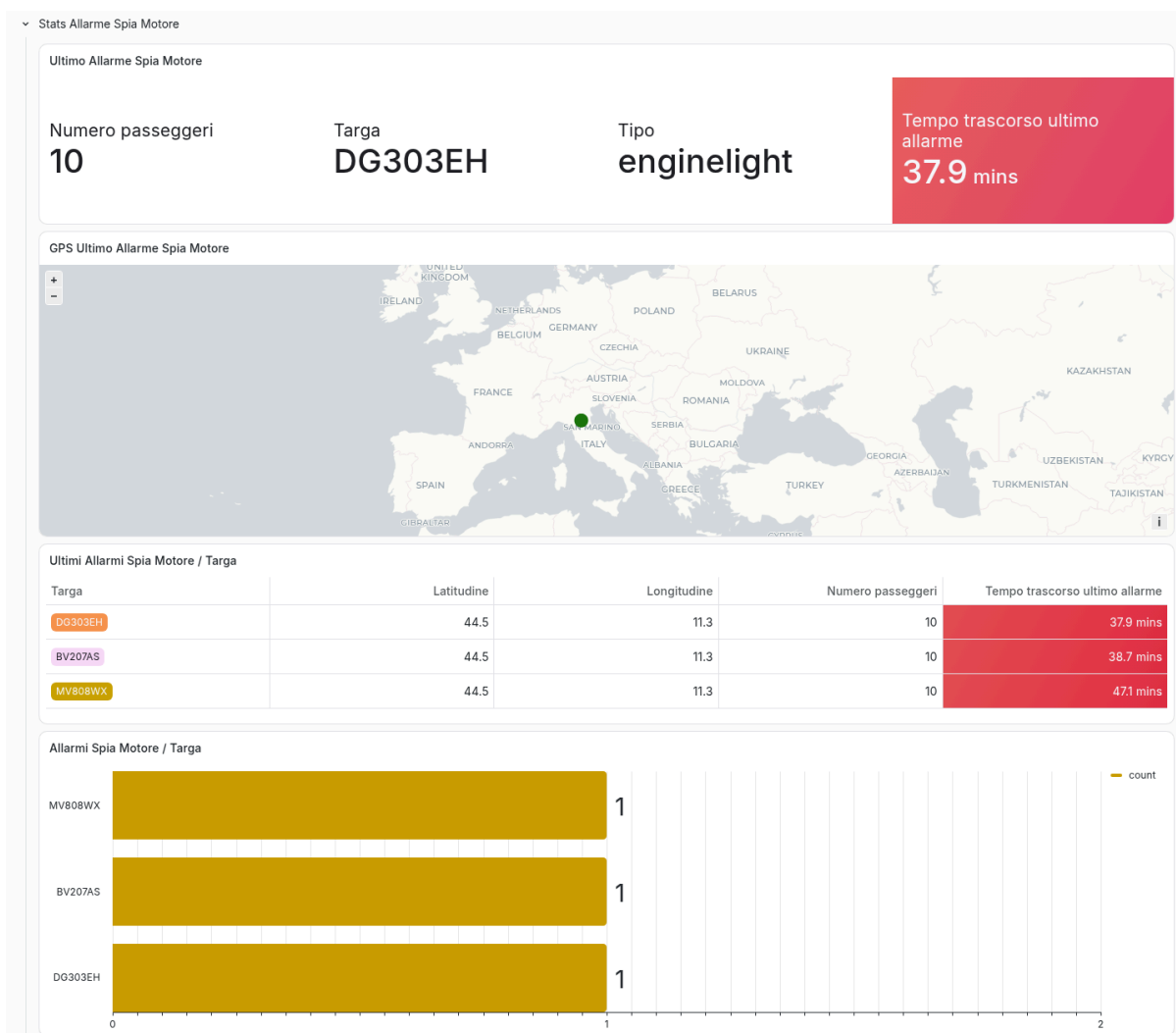


Figura 3.3. Immagine rappresentante il frammento di dashboard relativo alle informazioni associate all'allarme di spia motore.

Conclusioni e sviluppi futuri

L'attività sviluppata è stata la creazione di un sistema di monitoraggio in tempo reale di autobus, il processo di sviluppo ha visto in primo luogo lo studio e l'analisi di metriche che risultassero interessanti da osservare, del formato utilizzato per la comunicazione dei dati, necessariamente standardizzato, della frequenza di raccolta delle metriche, dimensionata ricercando un bilanciamento tra l'eccessivo e lo scarso invio di metriche, e del posizionamento del sistema centralizzato, avente lo scopo di accentrare su di sé la totalità delle informazioni raccolte, riportando lo studio di un caso reale. In secondo luogo lo sviluppo di un sistema in grado di simulare il funzionamento di un autobus e di comunicare la telemetria a un sistema centralizzato, la cui architettura organizzata a strati risulta suddivisa in due collocazioni differenti, gli strati di Ingestion e Processing collocati on-Edge, mentre gli strati di Storage e Visualizzazione in Cloud, con lo strato aggiuntivo di Integration frapposto tra gli autobus e l'edge del sistema.

L'Integration layer coinvolge l'utilizzo del protocollo MQTT, lo strato di Ingestion coinvolge il sistema di messaggistica Apache Kafka, il livello di Processing comprende l'utilizzo del framework Apache Flink, lo Storage layer coinvolge il DBMS NoSQL MongoDB, mentre lo strato finale di Visualizzazione comprende l'utilizzo di Grafana.

Il lavoro svolto ha permesso di centrare l'obiettivo prefissato, consentendo di implementare un sistema AVM per una qualsiasi società di trasporto pubblico, in grado di acquisire informazioni sulla telemetria prodotta da autobus, al fine di processare, memorizzare e visualizzare quest'ultima.

Il sistema potrebbe essere ampliato in futuro attraverso l'implementazione di load balancing per lo strato di Integrazione, in modo da poter ottimizzare il flusso e il carico da gestire per ogni messaggio da parte del suddetto layer. Un secondo sviluppo futuro è rappresentato da un'evoluzione dell'accodamento della telemetria osservata in caso di mancata comunicazione. Il logging degli eventi di sistema, e l'aumento della sicurezza attraverso l'utilizzo di un algoritmo di crittografia dei dati trasmessi dai veicoli, rappresentano ulteriori possibili sviluppi futuri.

Bibliografia

[IBM25] “Cos'è la telemetria?”, 2025. Reperibile presso:

<https://www.ibm.com/it-it/think/topics/telemetry>

[WIK01] “Automatic vehicle monitoring”, reperibile presso:

https://it.wikipedia.org/wiki/Automatic_vehicle_monitoring

[PLU01] “Quando il controllo diventa una leva di qualità del servizio”, reperibile

presso: <https://www.pluservice.net/it/soluzioni/monitoraggio-e-its-di-bordo>

[SMA01] “Una soluzione di monitoraggio della flotta del trasporto locale e di gestione delle aree a traffico limitato per un migliore servizio al cittadino: il caso Roma TPL”, reperibile presso:

<https://www.smau.it/casi-di-successo/una-soluzione-di-monitoraggio-della-flotta-del-trasporto-locale-e-di-gestione-delle-aree-a-traffico-limitato-per-un-migliore-servizio-al-cittadino-il-caso-roma-tpl>

[DAT25] Amber Jackson, “How Vodafone is Overhauling its Data Centre Infrastructure”, 2025. Reperibile presso:

<https://datacentremagazine.com/company-reports/how-vodafone-is-overhauling-its-data-centre-infrastructure>

[LEO21] “Leonardo: nuove soluzioni per trasporti sostenibili e sicuri”, 2021.

Reperibile presso:

<https://www.leonardo.com/it/news-and-stories-detail/-/detail/leonardo-innovative-solutions-for-safe-and-sustainable-transport>

[CRO01] “SISTEMA DI TELECONTROLLO PER LA GESTIONE DELLA FLOTTA”, reperibile presso:

https://www.cronacacomune.it/media/uploads/allegati/44/scheda_dettagliata_sistema_informativo_tper.pdf

[VOD01] “Vodafone Business Public and Private Cloud”, reperibile presso:

<https://www.vodafone.com/business/cloud-and-hosting/cloud-ready-it/move-to-cloud/public-and-private-cloud>

[MQT01] “MQTT vs. Kafka: Protocol Comparison for IoT and Data Streaming”, 2024.

Reperibile presso: <https://mqtt.pro/mqtt-vs-kafka/>

[CON01] “KRaft: Apache Kafka Without ZooKeeper”, reperibile presso:

<https://developer.confluent.io/learn/kraft/>

[KAF25] “AK 4.1.X”, 2025. Reperibile presso: <https://kafka.apache.org/41/>

[APA26] “Apache Flink Documentation”, 2026. Reperibile presso:

<https://nightlies.apache.org/flink/flink-docs-stable/>

[CON26] Stéphane Derosiaux, “Windowing in Apache Flink: Tumbling, Sliding, and Session Windows”, 2026. Reperibile presso:

<https://www.conduktor.io/glossary/windowing-in-apache-flink-tumbling-sliding-and-session-windows>

[MON01] “What is MongoDB Compass?”, reperibile presso:

<https://www.mongodb.com/docs/compass/current/>

[MED23] MetricFire, “What is Grafana?”, 2023. Reperibile presso:

<https://medium.com/@MetricFire/what-is-grafana-8de44d241765>

[ELE23] “Temperatura e capacità delle batterie auto elettriche”, 2023. Reperibile presso:

<https://electroverse.com/it-IT/community/ev-blogs-and-guides/come-influisce-la-temperatura-sulla-capacita-delle-batterie-ev>

[VOL01] “Specifiche Volvo 9700”, reperibile presso:

<https://www.volvobuses.com/it/coaches/coaches/volvo-9700/specifications.html>